

VEB-TREE-MINIMUM(V)

1 **return** $V.min$

VEB-TREE-MAXIMUM(V)

1 **return** $V.max$

Determining whether a value is in the set

The procedure VEB-TREE-MEMBER(V, x) has a recursive case like that of PROTO-VEB-MEMBER, but the base case is a little different. We also check directly whether x equals the minimum or maximum element. Since a vEB tree doesn't store bits as a proto-vEB structure does, we design VEB-TREE-MEMBER to return TRUE or FALSE rather than 1 or 0.

VEB-TREE-MEMBER(V, x)

1 **if** $x == V.min$ or $x == V.max$

2 **return** TRUE

3 **elseif** $V.u == 2$

4 **return** FALSE

5 **else return** VEB-TREE-MEMBER($V.cluster[high(x)], low(x)$)

Line 1 checks to see whether x equals either the minimum or maximum element. If it does, line 2 returns TRUE. Otherwise, line 3 tests for the base case. Since a $vEB(2)$ tree has no elements other than those in min and max , if it is the base case, line 4 returns FALSE. The other possibility—it is not a base case and x equals neither min nor max —is handled by the recursive call in line 5.

Recurrence (20.4) characterizes the running time of the VEB-TREE-MEMBER procedure, and so this procedure takes $O(\lg \lg u)$ time.

Finding the successor and predecessor

Next we see how to implement the SUCCESSOR operation. Recall that the procedure PROTO-VEB-SUCCESSOR(V, x) could make two recursive calls: one to determine whether x 's successor resides in the same cluster as x and, if it does not, one to find the cluster containing x 's successor. Because we can access the maximum value in a vEB tree quickly, we can avoid making two recursive calls, and instead make one recursive call on either a cluster or on the summary, but not on both.

```

VEB-TREE-SUCCESSOR( $V, x$ )
1  if  $V.u == 2$ 
2      if  $x == 0$  and  $V.max == 1$ 
3          return 1
4      else return NIL
5  elseif  $V.min \neq \text{NIL}$  and  $x < V.min$ 
6      return  $V.min$ 
7  else  $max-low = \text{VEB-TREE-MAXIMUM}(V.cluster[high(x)])$ 
8      if  $max-low \neq \text{NIL}$  and  $low(x) < max-low$ 
9           $offset = \text{VEB-TREE-SUCCESSOR}(V.cluster[high(x)], low(x))$ 
10         return  $\text{index}(high(x), offset)$ 
11     else  $succ-cluster = \text{VEB-TREE-SUCCESSOR}(V.summary, high(x))$ 
12         if  $succ-cluster == \text{NIL}$ 
13             return NIL
14         else  $offset = \text{VEB-TREE-MINIMUM}(V.cluster[succ-cluster])$ 
15         return  $\text{index}(succ-cluster, offset)$ 

```

This procedure has six **return** statements and several cases. We start with the base case in lines 2–4, which returns 1 in line 3 if we are trying to find the successor of 0 and 1 is in the 2-element set; otherwise, the base case returns NIL in line 4.

If we are not in the base case, we next check in line 5 whether x is strictly less than the minimum element. If so, then we simply return the minimum element in line 6.

If we get to line 7, then we know that we are not in a base case and that x is greater than or equal to the minimum value in the vEB tree V . Line 7 assigns to $max-low$ the maximum element in x 's cluster. If x 's cluster contains some element that is greater than x , then we know that x 's successor lies somewhere within x 's cluster. Line 8 tests for this condition. If x 's successor is within x 's cluster, then line 9 determines where in the cluster it is, and line 10 returns the successor in the same way as line 7 of PROTO-VEB-SUCCESSOR.

We get to line 11 if x is greater than or equal to the greatest element in its cluster. In this case, lines 11–15 find x 's successor in the same way as lines 8–12 of PROTO-VEB-SUCCESSOR.

It is easy to see how recurrence (20.4) characterizes the running time of VEB-TREE-SUCCESSOR. Depending on the result of the test in line 8, the procedure calls itself recursively in either line 9 (on a vEB tree with universe size $\sqrt[4]{u}$) or line 11 (on a vEB tree with universe size $\sqrt[4]{u}$). In either case, the one recursive call is on a vEB tree with universe size at most $\sqrt[4]{u}$. The remainder of the procedure, including the calls to VEB-TREE-MINIMUM and VEB-TREE-MAXIMUM, takes $O(1)$ time. Hence, VEB-TREE-SUCCESSOR runs in $O(\lg \lg u)$ worst-case time.

The `VEB-TREE-PREDECESSOR` procedure is symmetric to the `VEB-TREE-SUCCESSOR` procedure, but with one additional case:

```

VEB-TREE-PREDECESSOR(V, x)
1  if V.u == 2
2      if x == 1 and V.min == 0
3          return 0
4      else return NIL
5  elseif V.max ≠ NIL and x > V.max
6      return V.max
7  else min-low = VEB-TREE-MINIMUM(V.cluster[high(x)])
8      if min-low ≠ NIL and low(x) > min-low
9          offset = VEB-TREE-PREDECESSOR(V.cluster[high(x)], low(x))
10         return index(high(x), offset)
11     else pred-cluster = VEB-TREE-PREDECESSOR(V.summary, high(x))
12         if pred-cluster == NIL
13             if V.min ≠ NIL and x > V.min
14                 return V.min
15             else return NIL
16         else offset = VEB-TREE-MAXIMUM(V.cluster[pred-cluster])
17         return index(pred-cluster, offset)

```

Lines 13–14 form the additional case. This case occurs when x 's predecessor, if it exists, does not reside in x 's cluster. In `VEB-TREE-SUCCESSOR`, we were assured that if x 's successor resides outside of x 's cluster, then it must reside in a higher-numbered cluster. But if x 's predecessor is the minimum value in vEB tree V , then the predecessor resides in no cluster at all. Line 13 checks for this condition, and line 14 returns the minimum value as appropriate.

This extra case does not affect the asymptotic running time of `VEB-TREE-PREDECESSOR` when compared with `VEB-TREE-SUCCESSOR`, and so `VEB-TREE-PREDECESSOR` runs in $O(\lg \lg u)$ worst-case time.

Inserting an element

Now we examine how to insert an element into a vEB tree. Recall that `PROTO-VEB-INSERT` made two recursive calls: one to insert the element and one to insert the element's cluster number into the summary. The `VEB-TREE-INSERT` procedure will make only one recursive call. How can we get away with just one? When we insert an element, either the cluster that it goes into already has another element or it does not. If the cluster already has another element, then the cluster number is already in the summary, and so we do not need to make that recursive call. If

the cluster does not already have another element, then the element being inserted becomes the only element in the cluster, and we do not need to recurse to insert an element into an empty vEB tree:

VEB-EMPTY-TREE-INSERT(V, x)

```
1   $V.min = x$ 
2   $V.max = x$ 
```

With this procedure in hand, here is the pseudocode for VEB-TREE-INSERT(V, x), which assumes that x is not already an element in the set represented by vEB tree V :

VEB-TREE-INSERT(V, x)

```
1  if  $V.min == \text{NIL}$ 
2      VEB-EMPTY-TREE-INSERT( $V, x$ )
3  else if  $x < V.min$ 
4      exchange  $x$  with  $V.min$ 
5      if  $V.u > 2$ 
6          if VEB-TREE-MINIMUM( $V.cluster[\text{high}(x)]$ ) == NIL
7              VEB-TREE-INSERT( $V.summary, \text{high}(x)$ )
8              VEB-EMPTY-TREE-INSERT( $V.cluster[\text{high}(x)], \text{low}(x)$ )
9          else VEB-TREE-INSERT( $V.cluster[\text{high}(x)], \text{low}(x)$ )
10     if  $x > V.max$ 
11          $V.max = x$ 
```

This procedure works as follows. Line 1 tests whether V is an empty vEB tree and, if it is, then line 2 handles this easy case. Lines 3–11 assume that V is not empty, and therefore some element will be inserted into one of V 's clusters. But that element might not necessarily be the element x passed to VEB-TREE-INSERT. If $x < min$, as tested in line 3, then x needs to become the new min . We don't want to lose the original min , however, and so we need to insert it into one of V 's clusters. In this case, line 4 exchanges x with min , so that we insert the original min into one of V 's clusters.

We execute lines 6–9 only if V is not a base-case vEB tree. Line 6 determines whether the cluster that x will go into is currently empty. If so, then line 7 inserts x 's cluster number into the summary and line 8 handles the easy case of inserting x into an empty cluster. If x 's cluster is not currently empty, then line 9 inserts x into its cluster. In this case, we do not need to update the summary, since x 's cluster number is already a member of the summary.

Finally, lines 10–11 take care of updating max if $x > max$. Note that if V is a base-case vEB tree that is not empty, then lines 3–4 and 10–11 update min and max properly.

Once again, we can easily see how recurrence (20.4) characterizes the running time. Depending on the result of the test in line 6, either the recursive call in line 7 (run on a vEB tree with universe size $\sqrt[4]{u}$) or the recursive call in line 9 (run on a vEB with universe size $\sqrt[4]{u}$) executes. In either case, the one recursive call is on a vEB tree with universe size at most $\sqrt[4]{u}$. Because the remainder of VEB-TREE-INSERT takes $O(1)$ time, recurrence (20.4) applies, and so the running time is $O(\lg \lg u)$.

Deleting an element

Finally, we look at how to delete an element from a vEB tree. The procedure VEB-TREE-DELETE(V, x) assumes that x is currently an element in the set represented by the vEB tree V .

```

VEB-TREE-DELETE( $V, x$ )
1  if  $V.min == V.max$ 
2       $V.min = \text{NIL}$ 
3       $V.max = \text{NIL}$ 
4  elseif  $V.u == 2$ 
5      if  $x == 0$ 
6           $V.min = 1$ 
7      else  $V.min = 0$ 
8           $V.max = V.min$ 
9  else if  $x == V.min$ 
10      $first\_cluster = \text{VEB-TREE-MINIMUM}(V.summary)$ 
11      $x = \text{index}(first\_cluster,$ 
12          $\text{VEB-TREE-MINIMUM}(V.cluster[first\_cluster]))$ 
13      $V.min = x$ 
14     VEB-TREE-DELETE( $V.cluster[\text{high}(x)], \text{low}(x)$ )
15     if  $\text{VEB-TREE-MINIMUM}(V.cluster[\text{high}(x)]) == \text{NIL}$ 
16         VEB-TREE-DELETE( $V.summary, \text{high}(x)$ )
17     if  $x == V.max$ 
18          $summary\_max = \text{VEB-TREE-MAXIMUM}(V.summary)$ 
19         if  $summary\_max == \text{NIL}$ 
20              $V.max = V.min$ 
21         else  $V.max = \text{index}(summary\_max,$ 
22              $\text{VEB-TREE-MAXIMUM}(V.cluster[summary\_max]))$ 
23     elseif  $x == V.max$ 
24          $V.max = \text{index}(\text{high}(x),$ 
25              $\text{VEB-TREE-MAXIMUM}(V.cluster[\text{high}(x)]))$ 

```

The `VEB-TREE-DELETE` procedure works as follows. If the vEB tree V contains only one element, then it's just as easy to delete it as it was to insert an element into an empty vEB tree: just set min and max to `NIL`. Lines 1–3 handle this case. Otherwise, V has at least two elements. Line 4 tests whether V is a base-case vEB tree and, if so, lines 5–8 set min and max to the one remaining element.

Lines 9–22 assume that V has two or more elements and that $u \geq 4$. In this case, we will have to delete an element from a cluster. The element we delete from a cluster might not be x , however, because if x equals min , then once we have deleted x , some other element within one of V 's clusters becomes the new min , and we have to delete that other element from its cluster. If the test in line 9 reveals that we are in this case, then line 10 sets *first-cluster* to the number of the cluster that contains the lowest element other than min , and line 11 sets x to the value of the lowest element in that cluster. This element becomes the new min in line 12 and, because we set x to its value, it is the element that will be deleted from its cluster.

When we reach line 13, we know that we need to delete element x from its cluster, whether x was the value originally passed to `VEB-TREE-DELETE` or x is the element becoming the new minimum. Line 13 deletes x from its cluster. That cluster might now become empty, which line 14 tests, and if it does, then we need to remove x 's cluster number from the summary, which line 15 handles. After updating the summary, we might need to update max . Line 16 checks to see whether we are deleting the maximum element in V and, if we are, then line 17 sets *summary-max* to the number of the highest-numbered nonempty cluster. (The call `VEB-TREE-MAXIMUM( $V.summary$ )` works because we have already recursively called `VEB-TREE-DELETE` on $V.summary$, and therefore $V.summary.max$ has already been updated as necessary.) If all of V 's clusters are empty, then the only remaining element in V is min ; line 18 checks for this case, and line 19 updates max appropriately. Otherwise, line 20 sets max to the maximum element in the highest-numbered nonempty cluster. (If this cluster is where the element has been deleted, we again rely on the recursive call in line 13 having already corrected that cluster's max attribute.)

Finally, we have to handle the case in which x 's cluster did not become empty due to x being deleted. Although we do not have to update the summary in this case, we might have to update max . Line 21 tests for this case, and if we have to update max , line 22 does so (again relying on the recursive call to have corrected max in the cluster).

Now we show that `VEB-TREE-DELETE` runs in $O(\lg \lg u)$ time in the worst case. At first glance, you might think that recurrence (20.4) does not always apply, because a single call of `VEB-TREE-DELETE` can make two recursive calls: one on line 13 and one on line 15. Although the procedure can make both recursive calls, let's think about what happens when it does. In order for the recursive call on

line 15 to occur, the test on line 14 must show that x 's cluster is empty. The only way that x 's cluster can be empty is if x was the only element in its cluster when we made the recursive call on line 13. But if x was the only element in its cluster, then that recursive call took $O(1)$ time, because it executed only lines 1–3. Thus, we have two mutually exclusive possibilities:

- The recursive call on line 13 took constant time.
- The recursive call on line 15 did not occur.

In either case, recurrence (20.4) characterizes the running time of VEB-TREE-DELETE, and hence its worst-case running time is $O(\lg \lg u)$.

Exercises

20.3-1

Modify vEB trees to support duplicate keys.

20.3-2

Modify vEB trees to support keys that have associated satellite data.

20.3-3

Write pseudocode for a procedure that creates an empty van Emde Boas tree.

20.3-4

What happens if you call VEB-TREE-INSERT with an element that is already in the vEB tree? What happens if you call VEB-TREE-DELETE with an element that is not in the vEB tree? Explain why the procedures exhibit the behavior that they do. Show how to modify vEB trees and their operations so that we can check in constant time whether an element is present.

20.3-5

Suppose that instead of $\sqrt[k]{u}$ clusters, each with universe size $\sqrt[k]{u}$, we constructed vEB trees to have $u^{1/k}$ clusters, each with universe size $u^{1-1/k}$, where $k > 1$ is a constant. If we were to modify the operations appropriately, what would be their running times? For the purpose of analysis, assume that $u^{1/k}$ and $u^{1-1/k}$ are always integers.

20.3-6

Creating a vEB tree with universe size u requires $O(u)$ time. Suppose we wish to explicitly account for that time. What is the smallest number of operations n for which the amortized time of each operation in a vEB tree is $O(\lg \lg u)$?

Problems

20-1 Space requirements for van Emde Boas trees

This problem explores the space requirements for van Emde Boas trees and suggests a way to modify the data structure to make its space requirement depend on the number n of elements actually stored in the tree, rather than on the universe size u . For simplicity, assume that $\sqrt{u}$ is always an integer.

- a.* Explain why the following recurrence characterizes the space requirement $P(u)$ of a van Emde Boas tree with universe size u :

$$P(u) = (\sqrt{u} + 1)P(\sqrt{u}) + \Theta(\sqrt{u}). \quad (20.5)$$

- b.* Prove that recurrence (20.5) has the solution $P(u) = O(u)$.

In order to reduce the space requirements, let us define a **reduced-space van Emde Boas tree**, or **RS-vEB tree**, as a vEB tree V but with the following changes:

- The attribute $V.cluster$, rather than being stored as a simple array of pointers to vEB trees with universe size $\sqrt{u}$, is a hash table (see Chapter 11) stored as a dynamic table (see Section 17.4). Corresponding to the array version of $V.cluster$, the hash table stores pointers to RS-vEB trees with universe size $\sqrt{u}$. To find the i th cluster, we look up the key i in the hash table, so that we can find the i th cluster by a single search in the hash table.
- The hash table stores only pointers to nonempty clusters. A search in the hash table for an empty cluster returns NIL, indicating that the cluster is empty.
- The attribute $V.summary$ is NIL if all clusters are empty. Otherwise, $V.summary$ points to an RS-vEB tree with universe size $\sqrt{u}$.

Because the hash table is implemented with a dynamic table, the space it requires is proportional to the number of nonempty clusters.

When we need to insert an element into an empty RS-vEB tree, we create the RS-vEB tree by calling the following procedure, where the parameter u is the universe size of the RS-vEB tree:

CREATE-NEW-RS-vEB-TREE(u)

```

1  allocate a new vEB tree  $V$ 
2   $V.u = u$ 
3   $V.min = \text{NIL}$ 
4   $V.max = \text{NIL}$ 
5   $V.summary = \text{NIL}$ 
6  create  $V.cluster$  as an empty dynamic hash table
7  return  $V$ 
```


- c. Modify the `VEB-TREE-INSERT` procedure to produce pseudocode for the procedure `RS-VEB-TREE-INSERT( $V, x$ )`, which inserts x into the RS-VEB tree V , calling `CREATE-NEW-RS-VEB-TREE` as appropriate.
- d. Modify the `VEB-TREE-SUCCESSOR` procedure to produce pseudocode for the procedure `RS-VEB-TREE-SUCCESSOR( $V, x$ )`, which returns the successor of x in RS-VEB tree V , or `NIL` if x has no successor in V .
- e. Prove that, under the assumption of simple uniform hashing, your RS-VEB-TREE-INSERT and RS-VEB-TREE-SUCCESSOR procedures run in $O(\lg \lg u)$ expected amortized time.
- f. Assuming that elements are never deleted from a vEB tree, prove that the space requirement for the RS-VEB tree structure is $O(n)$, where n is the number of elements actually stored in the RS-VEB tree.
- g. RS-VEB trees have another advantage over vEB trees: they require less time to create. How long does it take to create an empty RS-VEB tree?

20-2 *y-fast tries*

This problem investigates D. Willard's "y-fast tries" which, like van Emde Boas trees, perform each of the operations `MEMBER`, `MINIMUM`, `MAXIMUM`, `PREDECESSOR`, and `SUCCESSOR` on elements drawn from a universe with size u in $O(\lg \lg u)$ worst-case time. The `INSERT` and `DELETE` operations take $O(\lg \lg u)$ amortized time. Like reduced-space van Emde Boas trees (see Problem 20-1), y-fast tries use only $O(n)$ space to store n elements. The design of y-fast tries relies on perfect hashing (see Section 11.5).

As a preliminary structure, suppose that we create a perfect hash table containing not only every element in the dynamic set, but every prefix of the binary representation of every element in the set. For example, if $u = 16$, so that $\lg u = 4$, and $x = 13$ is in the set, then because the binary representation of 13 is 1101, the perfect hash table would contain the strings 1, 11, 110, and 1101. In addition to the hash table, we create a doubly linked list of the elements currently in the set, in increasing order.

- a. How much space does this structure require?
- b. Show how to perform the `MINIMUM` and `MAXIMUM` operations in $O(1)$ time; the `MEMBER`, `PREDECESSOR`, and `SUCCESSOR` operations in $O(\lg \lg u)$ time; and the `INSERT` and `DELETE` operations in $O(\lg u)$ time.

To reduce the space requirement to $O(n)$, we make the following changes to the data structure:

- We cluster the n elements into $n / \lg u$ groups of size $\lg u$. (Assume for now that $\lg u$ divides n .) The first group consists of the $\lg u$ smallest elements in the set, the second group consists of the next $\lg u$ smallest elements, and so on.
- We designate a “representative” value for each group. The representative of the i th group is at least as large as the largest element in the i th group, and it is smaller than every element of the $(i + 1)$ st group. (The representative of the last group can be the maximum possible element $u - 1$.) Note that a representative might be a value not currently in the set.
- We store the $\lg u$ elements of each group in a balanced binary search tree, such as a red-black tree. Each representative points to the balanced binary search tree for its group, and each balanced binary search tree points to its group’s representative.
- The perfect hash table stores only the representatives, which are also stored in a doubly linked list in increasing order.

We call this structure a *y-fast trie*.

- c.* Show that a *y-fast trie* requires only $O(n)$ space to store n elements.
- d.* Show how to perform the MINIMUM and MAXIMUM operations in $O(\lg \lg u)$ time with a *y-fast trie*.
- e.* Show how to perform the MEMBER operation in $O(\lg \lg u)$ time.
- f.* Show how to perform the PREDECESSOR and SUCCESSOR operations in $O(\lg \lg u)$ time.
- g.* Explain why the INSERT and DELETE operations take $\Omega(\lg \lg u)$ time.
- h.* Show how to relax the requirement that each group in a *y-fast trie* has exactly $\lg u$ elements to allow INSERT and DELETE to run in $O(\lg \lg u)$ amortized time without affecting the asymptotic running times of the other operations.

Chapter notes

The data structure in this chapter is named after P. van Emde Boas, who described an early form of the idea in 1975 [339]. Later papers by van Emde Boas [340] and van Emde Boas, Kaas, and Zijlstra [341] refined the idea and the exposition. Mehlhorn and Näher [252] subsequently extended the ideas to apply to universe

sizes that are prime. Mehlhorn's book [249] contains a slightly different treatment of van Emde Boas trees than the one in this chapter.

Using the ideas behind van Emde Boas trees, Dementiev et al. [83] developed a nonrecursive, three-level search tree that ran faster than van Emde Boas trees in their own experiments.

Wang and Lin [347] designed a hardware-pipelined version of van Emde Boas trees, which achieves constant amortized time per operation and uses $O(\lg \lg u)$ stages in the pipeline.

A lower bound by Pătraşcu and Thorup [273, 274] for finding the predecessor shows that van Emde Boas trees are optimal for this operation, even if randomization is allowed.

21 Data Structures for Disjoint Sets

Some applications involve grouping n distinct elements into a collection of disjoint sets. These applications often need to perform two operations in particular: finding the unique set that contains a given element and uniting two sets. This chapter explores methods for maintaining a data structure that supports these operations.

Section 21.1 describes the operations supported by a disjoint-set data structure and presents a simple application. In Section 21.2, we look at a simple linked-list implementation for disjoint sets. Section 21.3 presents a more efficient representation using rooted trees. The running time using the tree representation is theoretically superlinear, but for all practical purposes it is linear. Section 21.4 defines and discusses a very quickly growing function and its very slowly growing inverse, which appears in the running time of operations on the tree-based implementation, and then, by a complex amortized analysis, proves an upper bound on the running time that is just barely superlinear.

21.1 Disjoint-set operations

A *disjoint-set data structure* maintains a collection $\mathcal{S} = \{S_1, S_2, \dots, S_k\}$ of disjoint dynamic sets. We identify each set by a *representative*, which is some member of the set. In some applications, it doesn't matter which member is used as the representative; we care only that if we ask for the representative of a dynamic set twice without modifying the set between the requests, we get the same answer both times. Other applications may require a prespecified rule for choosing the representative, such as choosing the smallest member in the set (assuming, of course, that the elements can be ordered).

As in the other dynamic-set implementations we have studied, we represent each element of a set by an object. Letting x denote an object, we wish to support the following operations:

MAKE-SET(x) creates a new set whose only member (and thus representative) is x . Since the sets are disjoint, we require that x not already be in some other set.

UNION(x, y) unites the dynamic sets that contain x and y , say S_x and S_y , into a new set that is the union of these two sets. We assume that the two sets are disjoint prior to the operation. The representative of the resulting set is any member of $S_x \cup S_y$, although many implementations of UNION specifically choose the representative of either S_x or S_y as the new representative. Since we require the sets in the collection to be disjoint, conceptually we destroy sets S_x and S_y , removing them from the collection $\mathcal{S}$. In practice, we often absorb the elements of one of the sets into the other set.

FIND-SET(x) returns a pointer to the representative of the (unique) set containing x .

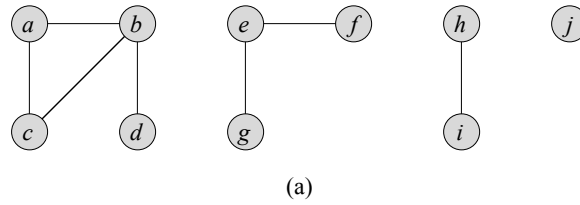
Throughout this chapter, we shall analyze the running times of disjoint-set data structures in terms of two parameters: n , the number of MAKE-SET operations, and m , the total number of MAKE-SET, UNION, and FIND-SET operations. Since the sets are disjoint, each UNION operation reduces the number of sets by one. After $n - 1$ UNION operations, therefore, only one set remains. The number of UNION operations is thus at most $n - 1$. Note also that since the MAKE-SET operations are included in the total number of operations m , we have $m \geq n$. We assume that the n MAKE-SET operations are the first n operations performed.

An application of disjoint-set data structures

One of the many applications of disjoint-set data structures arises in determining the connected components of an undirected graph (see Section B.4). Figure 21.1(a), for example, shows a graph with four connected components.

The procedure CONNECTED-COMPONENTS that follows uses the disjoint-set operations to compute the connected components of a graph. Once CONNECTED-COMPONENTS has preprocessed the graph, the procedure SAME-COMPONENT answers queries about whether two vertices are in the same connected component.¹ (In pseudocode, we denote the set of vertices of a graph G by $G.V$ and the set of edges by $G.E$.)

¹When the edges of the graph are static—not changing over time—we can compute the connected components faster by using depth-first search (Exercise 22.3-12). Sometimes, however, the edges are added dynamically and we need to maintain the connected components as each edge is added. In this case, the implementation given here can be more efficient than running a new depth-first search for each new edge.



Edge processed	Collection of disjoint sets									
initial sets	{a}	{b}	{c}	{d}	{e}	{f}	{g}	{h}	{i}	{j}
(b,d)	{a}	{b,d}	{c}		{e}	{f}	{g}	{h}	{i}	{j}
(e,g)	{a}	{b,d}	{c}		{e,g}	{f}		{h}	{i}	{j}
(a,c)	{a,c}	{b,d}			{e,g}	{f}		{h}	{i}	{j}
(h,i)	{a,c}	{b,d}			{e,g}	{f}		{h,i}		{j}
(a,b)	{a,b,c,d}				{e,g}	{f}		{h,i}		{j}
(e,f)	{a,b,c,d}				{e,f,g}			{h,i}		{j}
(b,c)	{a,b,c,d}				{e,f,g}			{h,i}		{j}

(b)

Figure 21.1 (a) A graph with four connected components: $\{a, b, c, d\}$, $\{e, f, g\}$, $\{h, i\}$, and $\{j\}$.
 (b) The collection of disjoint sets after processing each edge.

CONNECTED-COMPONENTS(G)

```

1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )

```

SAME-COMPONENT(u, v)

```

1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE

```

The procedure CONNECTED-COMPONENTS initially places each vertex v in its own set. Then, for each edge (u, v) , it unites the sets containing u and v . By Exercise 21.1-2, after processing all the edges, two vertices are in the same connected component if and only if the corresponding objects are in the same set. Thus, CONNECTED-COMPONENTS computes sets in such a way that the procedure SAME-COMPONENT can determine whether two vertices are in the same con-

nected component. Figure 21.1(b) illustrates how CONNECTED-COMPONENTS computes the disjoint sets.

In an actual implementation of this connected-components algorithm, the representations of the graph and the disjoint-set data structure would need to reference each other. That is, an object representing a vertex would contain a pointer to the corresponding disjoint-set object, and vice versa. These programming details depend on the implementation language, and we do not address them further here.

Exercises

21.1-1

Suppose that CONNECTED-COMPONENTS is run on the undirected graph $G = (V, E)$, where $V = \{a, b, c, d, e, f, g, h, i, j, k\}$ and the edges of E are processed in the order $(d, i), (f, k), (g, i), (b, g), (a, h), (i, j), (d, k), (b, j), (d, f), (g, j), (a, e)$. List the vertices in each connected component after each iteration of lines 3–5.

21.1-2

Show that after all edges are processed by CONNECTED-COMPONENTS, two vertices are in the same connected component if and only if they are in the same set.

21.1-3

During the execution of CONNECTED-COMPONENTS on an undirected graph $G = (V, E)$ with k connected components, how many times is FIND-SET called? How many times is UNION called? Express your answers in terms of $|V|$, $|E|$, and k .

21.2 Linked-list representation of disjoint sets

Figure 21.2(a) shows a simple way to implement a disjoint-set data structure: each set is represented by its own linked list. The object for each set has attributes *head*, pointing to the first object in the list, and *tail*, pointing to the last object. Each object in the list contains a set member, a pointer to the next object in the list, and a pointer back to the set object. Within each linked list, the objects may appear in any order. The representative is the set member in the first object in the list.

With this linked-list representation, both MAKE-SET and FIND-SET are easy, requiring $O(1)$ time. To carry out MAKE-SET(x), we create a new linked list whose only object is x . For FIND-SET(x), we just follow the pointer from x back to its set object and then return the member in the object that *head* points to. For example, in Figure 21.2(a), the call FIND-SET(g) would return f .

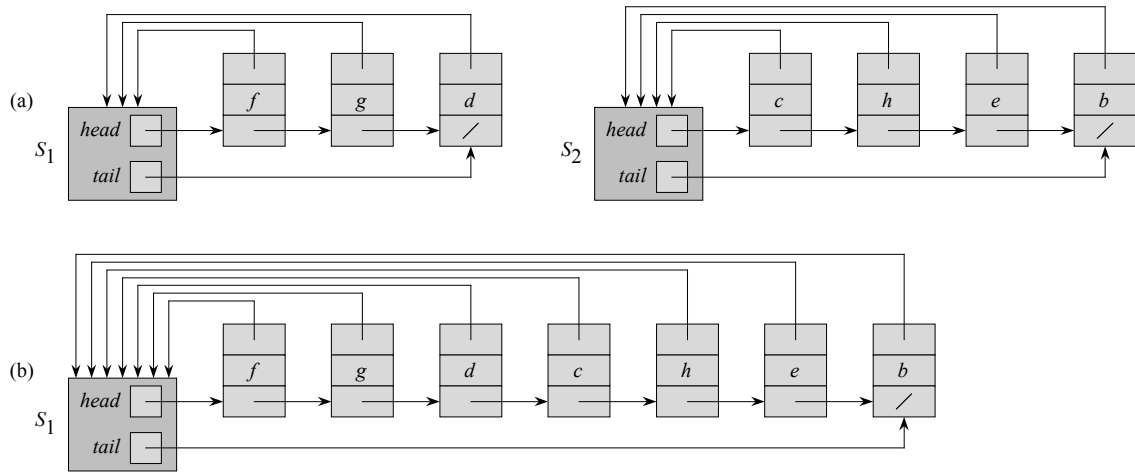


Figure 21.2 (a) Linked-list representations of two sets. Set S_1 contains members d , f , and g , with representative f , and set S_2 contains members b , c , e , and h , with representative c . Each object in the list contains a set member, a pointer to the next object in the list, and a pointer back to the set object. Each set object has pointers $head$ and $tail$ to the first and last objects, respectively. (b) The result of $\text{UNION}(g, e)$, which appends the linked list containing e to the linked list containing g . The representative of the resulting set is f . The set object for e 's list, S_2 , is destroyed.

A simple implementation of union

The simplest implementation of the UNION operation using the linked-list set representation takes significantly more time than MAKE-SET or FIND-SET . As Figure 21.2(b) shows, we perform $\text{UNION}(x, y)$ by appending y 's list onto the end of x 's list. The representative of x 's list becomes the representative of the resulting set. We use the $tail$ pointer for x 's list to quickly find where to append y 's list. Because all members of y 's list join x 's list, we can destroy the set object for y 's list. Unfortunately, we must update the pointer to the set object for each object originally on y 's list, which takes time linear in the length of y 's list. In Figure 21.2, for example, the operation $\text{UNION}(g, e)$ causes pointers to be updated in the objects for b , c , e , and h .

In fact, we can easily construct a sequence of m operations on n objects that requires $\Theta(n^2)$ time. Suppose that we have objects $x_1, x_2, \dots, x_n$. We execute the sequence of n MAKE-SET operations followed by $n - 1$ UNION operations shown in Figure 21.3, so that $m = 2n - 1$. We spend $\Theta(n)$ time performing the n MAKE-SET operations. Because the i th UNION operation updates i objects, the total number of objects updated by all $n - 1$ UNION operations is

Operation	Number of objects updated
MAKE-SET(x_1)	1
MAKE-SET(x_2)	1
$\vdots$	$\vdots$
MAKE-SET(x_n)	1
UNION(x_2, x_1)	1
UNION(x_3, x_2)	2
UNION(x_4, x_3)	3
$\vdots$	$\vdots$
UNION(x_n, x_{n-1})	$n - 1$

Figure 21.3 A sequence of $2n - 1$ operations on n objects that takes $\Theta(n^2)$ time, or $\Theta(n)$ time per operation on average, using the linked-list set representation and the simple implementation of UNION.

$$\sum_{i=1}^{n-1} i = \Theta(n^2) .$$

The total number of operations is $2n - 1$, and so each operation on average requires $\Theta(n)$ time. That is, the amortized time of an operation is $\Theta(n)$.

A weighted-union heuristic

In the worst case, the above implementation of the UNION procedure requires an average of $\Theta(n)$ time per call because we may be appending a longer list onto a shorter list; we must update the pointer to the set object for each member of the longer list. Suppose instead that each list also includes the length of the list (which we can easily maintain) and that we always append the shorter list onto the longer, breaking ties arbitrarily. With this simple *weighted-union heuristic*, a single UNION operation can still take $\Omega(n)$ time if both sets have $\Omega(n)$ members. As the following theorem shows, however, a sequence of m MAKE-SET, UNION, and FIND-SET operations, n of which are MAKE-SET operations, takes $O(m + n \lg n)$ time.

Theorem 21.1

Using the linked-list representation of disjoint sets and the weighted-union heuristic, a sequence of m MAKE-SET, UNION, and FIND-SET operations, n of which are MAKE-SET operations, takes $O(m + n \lg n)$ time.

Proof Because each UNION operation unites two disjoint sets, we perform at most $n - 1$ UNION operations over all. We now bound the total time taken by these UNION operations. We start by determining, for each object, an upper bound on the number of times the object's pointer back to its set object is updated. Consider a particular object x . We know that each time x 's pointer was updated, x must have started in the smaller set. The first time x 's pointer was updated, therefore, the resulting set must have had at least 2 members. Similarly, the next time x 's pointer was updated, the resulting set must have had at least 4 members. Continuing on, we observe that for any $k \leq n$, after x 's pointer has been updated $\lceil \lg k \rceil$ times, the resulting set must have at least k members. Since the largest set has at most n members, each object's pointer is updated at most $\lceil \lg n \rceil$ times over all the UNION operations. Thus the total time spent updating object pointers over all UNION operations is $O(n \lg n)$. We must also account for updating the *tail* pointers and the list lengths, which take only $\Theta(1)$ time per UNION operation. The total time spent in all UNION operations is thus $O(n \lg n)$.

The time for the entire sequence of m operations follows easily. Each MAKE-SET and FIND-SET operation takes $O(1)$ time, and there are $O(m)$ of them. The total time for the entire sequence is thus $O(m + n \lg n)$. ■

Exercises

21.2-1

Write pseudocode for MAKE-SET, FIND-SET, and UNION using the linked-list representation and the weighted-union heuristic. Make sure to specify the attributes that you assume for set objects and list objects.

21.2-2

Show the data structure that results and the answers returned by the FIND-SET operations in the following program. Use the linked-list representation with the weighted-union heuristic.

```

1  for  $i = 1$  to 16
2      MAKE-SET( $x_i$ )
3  for  $i = 1$  to 15 by 2
4      UNION( $x_i, x_{i+1}$ )
5  for  $i = 1$  to 13 by 4
6      UNION( $x_i, x_{i+2}$ )
7  UNION( $x_1, x_5$ )
8  UNION( $x_{11}, x_{13}$ )
9  UNION( $x_1, x_{10}$ )
10 FIND-SET( $x_2$ )
11 FIND-SET( $x_9$ )

```

Assume that if the sets containing x_i and x_j have the same size, then the operation $\text{UNION}(x_i, x_j)$ appends x_j 's list onto x_i 's list.

21.2-3

Adapt the aggregate proof of Theorem 21.1 to obtain amortized time bounds of $O(1)$ for MAKE-SET and FIND-SET and $O(\lg n)$ for UNION using the linked-list representation and the weighted-union heuristic.

21.2-4

Give a tight asymptotic bound on the running time of the sequence of operations in Figure 21.3 assuming the linked-list representation and the weighted-union heuristic.

21.2-5

Professor Gompers suspects that it might be possible to keep just one pointer in each set object, rather than two (*head* and *tail*), while keeping the number of pointers in each list element at two. Show that the professor's suspicion is well founded by describing how to represent each set by a linked list such that each operation has the same running time as the operations described in this section. Describe also how the operations work. Your scheme should allow for the weighted-union heuristic, with the same effect as described in this section. (*Hint*: Use the tail of a linked list as its set's representative.)

21.2-6

Suggest a simple change to the UNION procedure for the linked-list representation that removes the need to keep the *tail* pointer to the last object in each list. Whether or not the weighted-union heuristic is used, your change should not change the asymptotic running time of the UNION procedure. (*Hint*: Rather than appending one list to another, splice them together.)

21.3 Disjoint-set forests

In a faster implementation of disjoint sets, we represent sets by rooted trees, with each node containing one member and each tree representing one set. In a **disjoint-set forest**, illustrated in Figure 21.4(a), each member points only to its parent. The root of each tree contains the representative and is its own parent. As we shall see, although the straightforward algorithms that use this representation are no faster than ones that use the linked-list representation, by introducing two heuristics—"union by rank" and "path compression"—we can achieve an asymptotically optimal disjoint-set data structure.

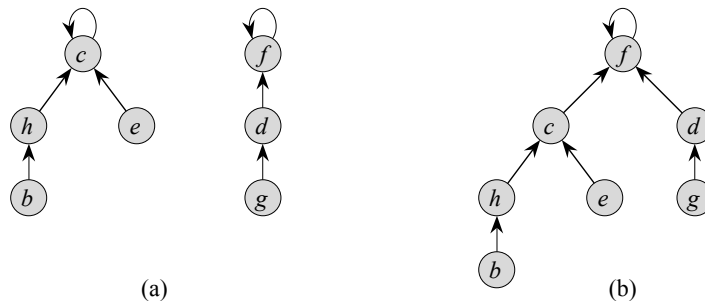


Figure 21.4 A disjoint-set forest. **(a)** Two trees representing the two sets of Figure 21.2. The tree on the left represents the set $\{b, c, e, h\}$, with c as the representative, and the tree on the right represents the set $\{d, f, g\}$, with f as the representative. **(b)** The result of $\text{UNION}(e, g)$.

We perform the three disjoint-set operations as follows. A **MAKE-SET** operation simply creates a tree with just one node. We perform a **FIND-SET** operation by following parent pointers until we find the root of the tree. The nodes visited on this simple path toward the root constitute the **find path**. A **UNION** operation, shown in Figure 21.4(b), causes the root of one tree to point to the root of the other.

Heuristics to improve the running time

So far, we have not improved on the linked-list implementation. A sequence of $n - 1$ **UNION** operations may create a tree that is just a linear chain of n nodes. By using two heuristics, however, we can achieve a running time that is almost linear in the total number of operations m .

The first heuristic, **union by rank**, is similar to the weighted-union heuristic we used with the linked-list representation. The obvious approach would be to make the root of the tree with fewer nodes point to the root of the tree with more nodes. Rather than explicitly keeping track of the size of the subtree rooted at each node, we shall use an approach that eases the analysis. For each node, we maintain a **rank**, which is an upper bound on the height of the node. In union by rank, we make the root with smaller rank point to the root with larger rank during a **UNION** operation.

The second heuristic, **path compression**, is also quite simple and highly effective. As shown in Figure 21.5, we use it during **FIND-SET** operations to make each node on the find path point directly to the root. Path compression does not change any ranks.

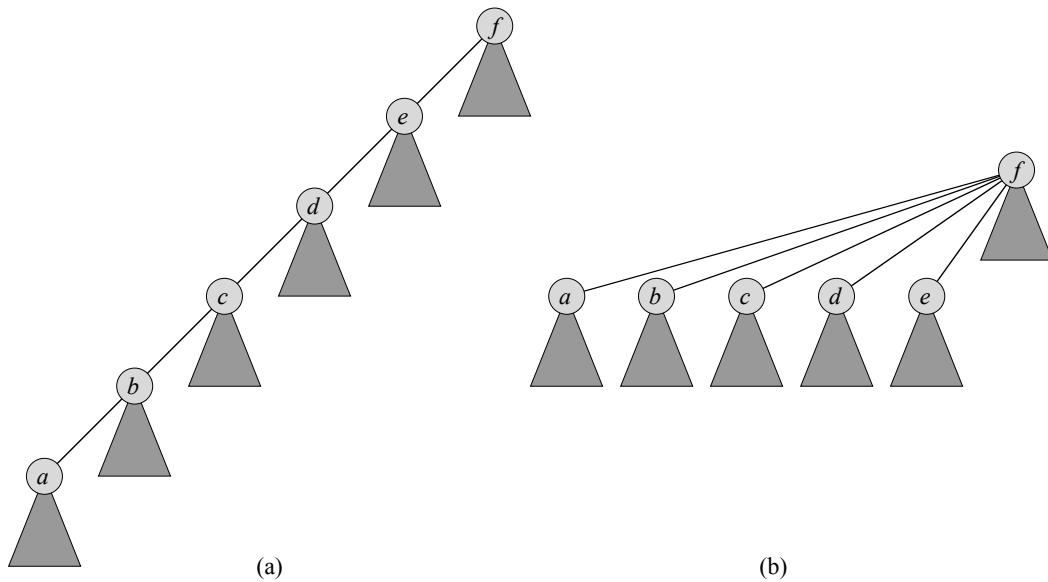


Figure 21.5 Path compression during the operation FIND-SET. Arrows and self-loops at roots are omitted. **(a)** A tree representing a set prior to executing FIND-SET(a). Triangles represent subtrees whose roots are the nodes shown. Each node has a pointer to its parent. **(b)** The same set after executing FIND-SET(a). Each node on the find path now points directly to the root.

Pseudocode for disjoint-set forests

To implement a disjoint-set forest with the union-by-rank heuristic, we must keep track of ranks. With each node x , we maintain the integer value $x.rank$, which is an upper bound on the height of x (the number of edges in the longest simple path from a descendant leaf to x). When MAKE-SET creates a singleton set, the single node in the corresponding tree has an initial rank of 0. Each FIND-SET operation leaves all ranks unchanged. The UNION operation has two cases, depending on whether the roots of the trees have equal rank. If the roots have unequal rank, we make the root with higher rank the parent of the root with lower rank, but the ranks themselves remain unchanged. If, instead, the roots have equal ranks, we arbitrarily choose one of the roots as the parent and increment its rank.

Let us put this method into pseudocode. We designate the parent of node x by $x.p$. The LINK procedure, a subroutine called by UNION, takes pointers to two roots as inputs.

MAKE-SET(x)

```
1   $x.p = x$ 
2   $x.rank = 0$ 
```

UNION(x, y)

```
1  LINK(FIND-SET( $x$ ), FIND-SET( $y$ ))
```

LINK(x, y)

```
1  if  $x.rank > y.rank$ 
2       $y.p = x$ 
3  else  $x.p = y$ 
4      if  $x.rank == y.rank$ 
5           $y.rank = y.rank + 1$ 
```

The FIND-SET procedure with path compression is quite simple:

FIND-SET(x)

```
1  if  $x \neq x.p$ 
2       $x.p = \text{FIND-SET}(x.p)$ 
3  return  $x.p$ 
```

The FIND-SET procedure is a *two-pass method*: as it recurses, it makes one pass up the find path to find the root, and as the recursion unwinds, it makes a second pass back down the find path to update each node to point directly to the root. Each call of FIND-SET(x) returns $x.p$ in line 3. If x is the root, then FIND-SET skips line 2 and instead returns $x.p$, which is x ; this is the case in which the recursion bottoms out. Otherwise, line 2 executes, and the recursive call with parameter $x.p$ returns a pointer to the root. Line 2 updates node x to point directly to the root, and line 3 returns this pointer.

Effect of the heuristics on the running time

Separately, either union by rank or path compression improves the running time of the operations on disjoint-set forests, and the improvement is even greater when we use the two heuristics together. Alone, union by rank yields a running time of $O(m \lg n)$ (see Exercise 21.4-4), and this bound is tight (see Exercise 21.3-3). Although we shall not prove it here, for a sequence of n MAKE-SET operations (and hence at most $n - 1$ UNION operations) and f FIND-SET operations, the path-compression heuristic alone gives a worst-case running time of $\Theta(n + f \cdot (1 + \log_{2+f/n} n))$.

When we use both union by rank and path compression, the worst-case running time is $O(m \alpha(n))$, where $\alpha(n)$ is a *very* slowly growing function, which we define in Section 21.4. In any conceivable application of a disjoint-set data structure, $\alpha(n) \leq 4$; thus, we can view the running time as linear in m in all practical situations. Strictly speaking, however, it is superlinear. In Section 21.4, we prove this upper bound.

Exercises

21.3-1

Redo Exercise 21.2-2 using a disjoint-set forest with union by rank and path compression.

21.3-2

Write a nonrecursive version of FIND-SET with path compression.

21.3-3

Give a sequence of m MAKE-SET, UNION, and FIND-SET operations, n of which are MAKE-SET operations, that takes $\Omega(m \lg n)$ time when we use union by rank only.

21.3-4

Suppose that we wish to add the operation PRINT-SET(x), which is given a node x and prints all the members of x 's set, in any order. Show how we can add just a single attribute to each node in a disjoint-set forest so that PRINT-SET(x) takes time linear in the number of members of x 's set and the asymptotic running times of the other operations are unchanged. Assume that we can print each member of the set in $O(1)$ time.

21.3-5 ★

Show that any sequence of m MAKE-SET, FIND-SET, and LINK operations, where all the LINK operations appear before any of the FIND-SET operations, takes only $O(m)$ time if we use both path compression and union by rank. What happens in the same situation if we use only the path-compression heuristic?

★ 21.4 Analysis of union by rank with path compression

As noted in Section 21.3, the combined union-by-rank and path-compression heuristic runs in time $O(m \alpha(n))$ for m disjoint-set operations on n elements. In this section, we shall examine the function α to see just how slowly it grows. Then we prove this running time using the potential method of amortized analysis.

A very quickly growing function and its very slowly growing inverse

For integers $k \geq 0$ and $j \geq 1$, we define the function $A_k(j)$ as

$$A_k(j) = \begin{cases} j + 1 & \text{if } k = 0, \\ A_{k-1}^{(j+1)}(j) & \text{if } k \geq 1, \end{cases}$$

where the expression $A_{k-1}^{(j+1)}(j)$ uses the functional-iteration notation given in Section 3.2. Specifically, $A_{k-1}^{(0)}(j) = j$ and $A_{k-1}^{(i)}(j) = A_{k-1}(A_{k-1}^{(i-1)}(j))$ for $i \geq 1$. We will refer to the parameter k as the **level** of the function A .

The function $A_k(j)$ strictly increases with both j and k . To see just how quickly this function grows, we first obtain closed-form expressions for $A_1(j)$ and $A_2(j)$.

Lemma 21.2

For any integer $j \geq 1$, we have $A_1(j) = 2j + 1$.

Proof We first use induction on i to show that $A_0^{(i)}(j) = j + i$. For the base case, we have $A_0^{(0)}(j) = j = j + 0$. For the inductive step, assume that $A_0^{(i-1)}(j) = j + (i - 1)$. Then $A_0^{(i)}(j) = A_0(A_0^{(i-1)}(j)) = (j + (i - 1)) + 1 = j + i$. Finally, we note that $A_1(j) = A_0^{(j+1)}(j) = j + (j + 1) = 2j + 1$. ■

Lemma 21.3

For any integer $j \geq 1$, we have $A_2(j) = 2^{j+1}(j + 1) - 1$.

Proof We first use induction on i to show that $A_1^{(i)}(j) = 2^i(j + 1) - 1$. For the base case, we have $A_1^{(0)}(j) = j = 2^0(j + 1) - 1$. For the inductive step, assume that $A_1^{(i-1)}(j) = 2^{i-1}(j + 1) - 1$. Then $A_1^{(i)}(j) = A_1(A_1^{(i-1)}(j)) = A_1(2^{i-1}(j + 1) - 1) = 2 \cdot (2^{i-1}(j + 1) - 1) + 1 = 2^i(j + 1) - 2 + 1 = 2^i(j + 1) - 1$. Finally, we note that $A_2(j) = A_1^{(j+1)}(j) = 2^{j+1}(j + 1) - 1$. ■

Now we can see how quickly $A_k(j)$ grows by simply examining $A_k(1)$ for levels $k = 0, 1, 2, 3, 4$. From the definition of $A_0(k)$ and the above lemmas, we have $A_0(1) = 1 + 1 = 2$, $A_1(1) = 2 \cdot 1 + 1 = 3$, and $A_2(1) = 2^{1+1} \cdot (1 + 1) - 1 = 7$.

We also have

$$\begin{aligned}
 A_3(1) &= A_2^{(2)}(1) \\
 &= A_2(A_2(1)) \\
 &= A_2(7) \\
 &= 2^8 \cdot 8 - 1 \\
 &= 2^{11} - 1 \\
 &= 2047
 \end{aligned}$$

and

$$\begin{aligned}
 A_4(1) &= A_3^{(2)}(1) \\
 &= A_3(A_3(1)) \\
 &= A_3(2047) \\
 &= A_2^{(2048)}(2047) \\
 &\gg A_2(2047) \\
 &= 2^{2048} \cdot 2048 - 1 \\
 &> 2^{2048} \\
 &= (2^4)^{512} \\
 &= 16^{512} \\
 &\gg 10^{80} ,
 \end{aligned}$$

which is the estimated number of atoms in the observable universe. (The symbol “ $\gg$ ” denotes the “much-greater-than” relation.)

We define the inverse of the function $A_k(n)$, for integer $n \geq 0$, by

$$\alpha(n) = \min \{k : A_k(1) \geq n\} .$$

In words, $\alpha(n)$ is the lowest level k for which $A_k(1)$ is at least n . From the above values of $A_k(1)$, we see that

$$\alpha(n) = \begin{cases} 0 & \text{for } 0 \leq n \leq 2 , \\ 1 & \text{for } n = 3 , \\ 2 & \text{for } 4 \leq n \leq 7 , \\ 3 & \text{for } 8 \leq n \leq 2047 , \\ 4 & \text{for } 2048 \leq n \leq A_4(1) . \end{cases}$$

It is only for values of n so large that the term “astronomical” understates them (greater than $A_4(1)$, a huge number) that $\alpha(n) > 4$, and so $\alpha(n) \leq 4$ for all practical purposes.

Properties of ranks

In the remainder of this section, we prove an $O(m\alpha(n))$ bound on the running time of the disjoint-set operations with union by rank and path compression. In order to prove this bound, we first prove some simple properties of ranks.

Lemma 21.4

For all nodes x , we have $x.rank \leq x.p.rank$, with strict inequality if $x \neq x.p$. The value of $x.rank$ is initially 0 and increases through time until $x \neq x.p$; from then on, $x.rank$ does not change. The value of $x.p.rank$ monotonically increases over time.

Proof The proof is a straightforward induction on the number of operations, using the implementations of MAKE-SET, UNION, and FIND-SET that appear in Section 21.3. We leave it as Exercise 21.4-1. ■

Corollary 21.5

As we follow the simple path from any node toward a root, the node ranks strictly increase. ■

Lemma 21.6

Every node has rank at most $n - 1$.

Proof Each node's rank starts at 0, and it increases only upon LINK operations. Because there are at most $n - 1$ UNION operations, there are also at most $n - 1$ LINK operations. Because each LINK operation either leaves all ranks alone or increases some node's rank by 1, all ranks are at most $n - 1$. ■

Lemma 21.6 provides a weak bound on ranks. In fact, every node has rank at most $\lceil \lg n \rceil$ (see Exercise 21.4-2). The looser bound of Lemma 21.6 will suffice for our purposes, however.

Proving the time bound

We shall use the potential method of amortized analysis (see Section 17.3) to prove the $O(m\alpha(n))$ time bound. In performing the amortized analysis, we will find it convenient to assume that we invoke the LINK operation rather than the UNION operation. That is, since the parameters of the LINK procedure are pointers to two roots, we act as though we perform the appropriate FIND-SET operations separately. The following lemma shows that even if we count the extra FIND-SET operations induced by UNION calls, the asymptotic running time remains unchanged.

Lemma 21.7

Suppose we convert a sequence S' of m' MAKE-SET, UNION, and FIND-SET operations into a sequence S of m MAKE-SET, LINK, and FIND-SET operations by turning each UNION into two FIND-SET operations followed by a LINK. Then, if sequence S runs in $O(m \alpha(n))$ time, sequence S' runs in $O(m' \alpha(n))$ time.

Proof Since each UNION operation in sequence S' is converted into three operations in S , we have $m' \leq m \leq 3m'$. Since $m = O(m')$, an $O(m \alpha(n))$ time bound for the converted sequence S implies an $O(m' \alpha(n))$ time bound for the original sequence S' . ■

In the remainder of this section, we shall assume that the initial sequence of m' MAKE-SET, UNION, and FIND-SET operations has been converted to a sequence of m MAKE-SET, LINK, and FIND-SET operations. We now prove an $O(m \alpha(n))$ time bound for the converted sequence and appeal to Lemma 21.7 to prove the $O(m' \alpha(n))$ running time of the original sequence of m' operations.

Potential function

The potential function we use assigns a potential $\phi_q(x)$ to each node x in the disjoint-set forest after q operations. We sum the node potentials for the potential of the entire forest: $\Phi_q = \sum_x \phi_q(x)$, where Φ_q denotes the potential of the forest after q operations. The forest is empty prior to the first operation, and we arbitrarily set $\Phi_0 = 0$. No potential Φ_q will ever be negative.

The value of $\phi_q(x)$ depends on whether x is a tree root after the q th operation. If it is, or if $x.rank = 0$, then $\phi_q(x) = \alpha(n) \cdot x.rank$.

Now suppose that after the q th operation, x is not a root and that $x.rank \geq 1$. We need to define two auxiliary functions on x before we can define $\phi_q(x)$. First we define

$$\text{level}(x) = \max \{k : x.p.rank \geq A_k(x.rank)\} .$$

That is, $\text{level}(x)$ is the greatest level k for which A_k , applied to x 's rank, is no greater than x 's parent's rank.

We claim that

$$0 \leq \text{level}(x) < \alpha(n) , \tag{21.1}$$

which we see as follows. We have

$$\begin{aligned} x.p.rank &\geq x.rank + 1 \quad (\text{by Lemma 21.4}) \\ &= A_0(x.rank) \quad (\text{by definition of } A_0(j)) , \end{aligned}$$

which implies that $\text{level}(x) \geq 0$, and we have

$$\begin{aligned}
A_{\alpha(n)}(x.rank) &\geq A_{\alpha(n)}(1) && \text{(because } A_k(j) \text{ is strictly increasing)} \\
&\geq n && \text{(by the definition of } \alpha(n)) \\
&> x.p.rank && \text{(by Lemma 21.6) ,}
\end{aligned}$$

which implies that $\text{level}(x) < \alpha(n)$. Note that because $x.p.rank$ monotonically increases over time, so does $\text{level}(x)$.

The second auxiliary function applies when $x.rank \geq 1$:

$$\text{iter}(x) = \max \{i : x.p.rank \geq A_{\text{level}(x)}^{(i)}(x.rank)\} .$$

That is, $\text{iter}(x)$ is the largest number of times we can iteratively apply $A_{\text{level}(x)}$, applied initially to x 's rank, before we get a value greater than x 's parent's rank.

We claim that when $x.rank \geq 1$, we have

$$1 \leq \text{iter}(x) \leq x.rank , \tag{21.2}$$

which we see as follows. We have

$$\begin{aligned}
x.p.rank &\geq A_{\text{level}(x)}(x.rank) && \text{(by definition of } \text{level}(x)) \\
&= A_{\text{level}(x)}^{(1)}(x.rank) && \text{(by definition of functional iteration) ,}
\end{aligned}$$

which implies that $\text{iter}(x) \geq 1$, and we have

$$\begin{aligned}
A_{\text{level}(x)}^{(x.rank+1)}(x.rank) &= A_{\text{level}(x)+1}(x.rank) && \text{(by definition of } A_k(j)) \\
&> x.p.rank && \text{(by definition of } \text{level}(x)) ,
\end{aligned}$$

which implies that $\text{iter}(x) \leq x.rank$. Note that because $x.p.rank$ monotonically increases over time, in order for $\text{iter}(x)$ to decrease, $\text{level}(x)$ must increase. As long as $\text{level}(x)$ remains unchanged, $\text{iter}(x)$ must either increase or remain unchanged.

With these auxiliary functions in place, we are ready to define the potential of node x after q operations:

$$\phi_q(x) = \begin{cases} \alpha(n) \cdot x.rank & \text{if } x \text{ is a root or } x.rank = 0 , \\ (\alpha(n) - \text{level}(x)) \cdot x.rank - \text{iter}(x) & \text{if } x \text{ is not a root and } x.rank \geq 1 . \end{cases}$$

We next investigate some useful properties of node potentials.

Lemma 21.8

For every node x , and for all operation counts q , we have

$$0 \leq \phi_q(x) \leq \alpha(n) \cdot x.rank .$$

Proof If x is a root or $x.rank = 0$, then $\phi_q(x) = \alpha(n) \cdot x.rank$ by definition. Now suppose that x is not a root and that $x.rank \geq 1$. We obtain a lower bound on $\phi_q(x)$ by maximizing $level(x)$ and $iter(x)$. By the bound (21.1), $level(x) \leq \alpha(n) - 1$, and by the bound (21.2), $iter(x) \leq x.rank$. Thus,

$$\begin{aligned} \phi_q(x) &= (\alpha(n) - level(x)) \cdot x.rank - iter(x) \\ &\geq (\alpha(n) - (\alpha(n) - 1)) \cdot x.rank - x.rank \\ &= x.rank - x.rank \\ &= 0. \end{aligned}$$

Similarly, we obtain an upper bound on $\phi_q(x)$ by minimizing $level(x)$ and $iter(x)$. By the bound (21.1), $level(x) \geq 0$, and by the bound (21.2), $iter(x) \geq 1$. Thus,

$$\begin{aligned} \phi_q(x) &\leq (\alpha(n) - 0) \cdot x.rank - 1 \\ &= \alpha(n) \cdot x.rank - 1 \\ &< \alpha(n) \cdot x.rank. \end{aligned}$$

■

Corollary 21.9

If node x is not a root and $x.rank > 0$, then $\phi_q(x) < \alpha(n) \cdot x.rank$.

■

Potential changes and amortized costs of operations

We are now ready to examine how the disjoint-set operations affect node potentials. With an understanding of the change in potential due to each operation, we can determine each operation's amortized cost.

Lemma 21.10

Let x be a node that is not a root, and suppose that the q th operation is either a LINK or FIND-SET. Then after the q th operation, $\phi_q(x) \leq \phi_{q-1}(x)$. Moreover, if $x.rank \geq 1$ and either $level(x)$ or $iter(x)$ changes due to the q th operation, then $\phi_q(x) \leq \phi_{q-1}(x) - 1$. That is, x 's potential cannot increase, and if it has positive rank and either $level(x)$ or $iter(x)$ changes, then x 's potential drops by at least 1.

Proof Because x is not a root, the q th operation does not change $x.rank$, and because n does not change after the initial n MAKE-SET operations, $\alpha(n)$ remains unchanged as well. Hence, these components of the formula for x 's potential remain the same after the q th operation. If $x.rank = 0$, then $\phi_q(x) = \phi_{q-1}(x) = 0$. Now assume that $x.rank \geq 1$.

Recall that $level(x)$ monotonically increases over time. If the q th operation leaves $level(x)$ unchanged, then $iter(x)$ either increases or remains unchanged. If both $level(x)$ and $iter(x)$ are unchanged, then $\phi_q(x) = \phi_{q-1}(x)$. If $level(x)$

is unchanged and $\text{iter}(x)$ increases, then it increases by at least 1, and so $\phi_q(x) \leq \phi_{q-1}(x) - 1$.

Finally, if the q th operation increases $\text{level}(x)$, it increases by at least 1, so that the value of the term $(\alpha(n) - \text{level}(x)) \cdot x.\text{rank}$ drops by at least $x.\text{rank}$. Because $\text{level}(x)$ increased, the value of $\text{iter}(x)$ might drop, but according to the bound (21.2), the drop is by at most $x.\text{rank} - 1$. Thus, the increase in potential due to the change in $\text{iter}(x)$ is less than the decrease in potential due to the change in $\text{level}(x)$, and we conclude that $\phi_q(x) \leq \phi_{q-1}(x) - 1$. ■

Our final three lemmas show that the amortized cost of each MAKE-SET, LINK, and FIND-SET operation is $O(\alpha(n))$. Recall from equation (17.2) that the amortized cost of each operation is its actual cost plus the increase in potential due to the operation.

Lemma 21.11

The amortized cost of each MAKE-SET operation is $O(1)$.

Proof Suppose that the q th operation is MAKE-SET(x). This operation creates node x with rank 0, so that $\phi_q(x) = 0$. No other ranks or potentials change, and so $\Phi_q = \Phi_{q-1}$. Noting that the actual cost of the MAKE-SET operation is $O(1)$ completes the proof. ■

Lemma 21.12

The amortized cost of each LINK operation is $O(\alpha(n))$.

Proof Suppose that the q th operation is LINK(x, y). The actual cost of the LINK operation is $O(1)$. Without loss of generality, suppose that the LINK makes y the parent of x .

To determine the change in potential due to the LINK, we note that the only nodes whose potentials may change are x , y , and the children of y just prior to the operation. We shall show that the only node whose potential can increase due to the LINK is y , and that its increase is at most $\alpha(n)$:

- By Lemma 21.10, any node that is y 's child just before the LINK cannot have its potential increase due to the LINK.
- From the definition of $\phi_q(x)$, we see that, since x was a root just before the q th operation, $\phi_{q-1}(x) = \alpha(n) \cdot x.\text{rank}$. If $x.\text{rank} = 0$, then $\phi_q(x) = \phi_{q-1}(x) = 0$. Otherwise,

$$\begin{aligned} \phi_q(x) &< \alpha(n) \cdot x.\text{rank} \quad (\text{by Corollary 21.9}) \\ &= \phi_{q-1}(x), \end{aligned}$$

and so x 's potential decreases.